

Safe integral comparisons

I. Table of Contents

- [I\).....Table of Contents](#)
- [II\).....Motivation](#)
- [III\).....Proposal](#)
- [IV\).....Examples](#)
- [V\).....Possible implementation](#)
- [VI\).....Effects on Existing Code](#)
- [VII\).....Design Decisions](#)
- [VIII\).....Related Works](#)

II. Motivation

Comparing integrals of different types may be a more complex task than expected. Most of the time we expect that a simple

```
if(a < b){  
    // ...  
} else {  
    // ...  
}
```

should work in all cases, but if `a` and `b` are of different types, things are more complicated.

If `a` is a signed type, and `b` unsigned, then `a` is converted to the unsigned type. If `a` held a number less than zero, then the result may be unexpected, since the expression `a < b` could evaluate to false, even if `a` strictly negative number is always lower than a positive one.

Also converting integrals between different types can be challenging, for simplicity, most of the time we assume that values are in range, and write

```
a = static_cast<decltype(a)>(b);
```

If we want to write a safe conversion, we need to check if `b` has a value between `std::numeric_limits<decltype(a)>::min()` and `std::numeric_limits<decltype(a)>::max()`. We also need to pay attention that no implicit conversion (for example between unsigned and signed types) invalidates our comparison.

Comparing and converting numbers, even of different numeric types, should be a trivial task. Unfortunately it is not, and because of implicit conversions we may write, without noticing it, unsafe code.

III. Proposal

This paper proposes to add a set of `constexpr` and `noexcept` functions for converting and comparing integrals of different signedness (except for `bool`):

- Two functions to compare if two variables represent the same value or not

```
template <typename T, typename U>  
constexpr bool std::cmp_equal(T t, U u) noexcept;  
  
template <typename T, typename U>  
constexpr bool std::cmp_unequal(T t, U u) noexcept;
```

- A set of functions that can be used to determine the relative order of two values

```
template <typename T, typename U>  
constexpr bool std::cmp_less(T t, U u) noexcept;  
  
template <typename T, typename U>  
constexpr bool std::cmp_greater(T t, U u) noexcept;  
  
template <typename T, typename U>  
constexpr bool std::cmp_less_or_equal(T t, U u) noexcept;  
  
template <typename T, typename U>  
constexpr bool std::cmp_greater_or_equal(T t, U u) noexcept;
```

- One function to determine if a specific value is inside the range of possible values of another type (i.e. if we can convert the value to the other type safely)

```
template <typename R, typename T>  
constexpr bool in_range(T t) noexcept;
```

IV. Examples

Examples without current proposal

Comparing an unsigned int with an int:

```
int a = ...  
unsigned int b = ...  
// added static_cast to avoid compiler warnings since we are doing a "safe" comparison  
if(a < 0 || static_cast<unsigned int>(a) < b){  
    // do X  
} else {  
    // do Y  
}
```

Comparing an `uint32_t` with an `int16_t`:

```
int32_t a = ...  
uint16_t b = ...  
// added static_cast to avoid compiler warnings since we are doing a "safe" comparison  
if(a < static_cast<int32_t>(b)){  
    // do X  
} else {  
    // do Y  
}
```

```
}
```

Comparing an int with an intptr_t:

```
int a = ...
intptr_t b = ...
if(???){ // no idea how to do it in one readable line without some assumption about int and intptr_t
    // do X
} else {
    // do Y
}
```

Example with current proposal

Comparing one integral type A with another integral type B (both non bool):

```
A a = ...
B b = ...
// no need for any cast since std::cmp_less is taking care of everything
if( std::cmp_less(a,b)){
    // do X
} else {
    // do Y
}
```

V. Possible implementation

This section shows an example of how `cmp_equal`, `cmp_less` and `in_range` can be implemented with any standard conforming C++11 compiler. The only dependencies are the `std::numeric_limits` function from the `limits` header and some traits from the `type_traits` header. This implementation can also be found on [github](#).

```
#include <limits>
#include <type_traits>

namespace details{
#if defined(ERR_MSG_xxx_NEEDS_INTEGRAL_NOT_BOOL) || defined(ASSERT_INTEGRAL_NOT_BOOL_TYPE)
#error "ERR_MSG_xxx_NEEDS_INTEGRAL_NOT_BOOL or ASSERT_INTEGRAL_NOT_BOOL_TYPE already defined"
#endif
#define ERR_MSG_xxx_NEEDS_INTEGRAL_NOT_BOOL " needs to be an integral (not bool) value type"
#define ASSERT_INTEGRAL_NOT_BOOL_TYPE(T) static_assert(is_integral_not_bool<T>(), #T ERR_MSG_xxx_NEEDS_INTEGRAL_NOT_BOOL);

template <typename T>
constexpr bool is_integral_not_bool(){
    using value_type = typename std::remove_cv<T>::type;
    return !std::is_same<value_type,bool>::value && std::is_integral<T>::value;
}

// could use the same implementation of in_range_signed_signed, but compiler may generate warning that t is always bigger than 0
template <typename R, typename T>
constexpr bool in_range_unsigned_unsigned(const T t) noexcept {
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(T);
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(R);
    return (std::numeric_limits<T>::digits > std::numeric_limits<R>::digits) ?
        (t < static_cast<T>(std::numeric_limits<R>::max())) :
        (static_cast<R>(t) < std::numeric_limits<R>::max());
}

template <typename R, typename T>
constexpr bool in_range_signed_signed(const T t) noexcept {
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(T);
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(R);
    return (std::numeric_limits<T>::digits > std::numeric_limits<R>::digits) ?
        (t <= static_cast<T>(std::numeric_limits<R>::max()) && t >= static_cast<T>(std::numeric_limits<R>::min())) :
        (static_cast<R>(t) <= std::numeric_limits<R>::max() && static_cast<R>(t) >= std::numeric_limits<R>::min());
}

template <typename R, typename T>
constexpr bool in_range_signed_unsigned(const T t) noexcept {
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(T);
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(R);
    return (t < T{ 0 }) ? false :
        (std::numeric_limits<T>::digits / 2 <= std::numeric_limits<R>::digits) ? true :
        (t <= static_cast<T>(std::numeric_limits<R>::max()));
}

template <typename R, typename T>
constexpr bool in_range_unsigned_signed(const T t) noexcept {
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(T);
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(R);
    return (std::numeric_limits<T>::digits >= std::numeric_limits<R>::digits / 2) ? (t <= static_cast<T>(std::numeric_limits<R>::max())) : true;
}

template <typename R, typename T>
constexpr bool in_range_unsigned(const T t) noexcept {
    return std::is_unsigned<R>::value ? in_range_unsigned_unsigned<R>(t) : in_range_unsigned_signed<R>(t);
}

template <typename R, typename T>
constexpr bool in_range_signed(const T t) noexcept {
    return std::is_signed<R>::value ? in_range_signed_signed<R>(t) : in_range_signed_unsigned<R>(t);
}

template <typename T, typename U>
constexpr bool cmp_equal_same_sign(const T t, const U u) noexcept {
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(T);
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(U);
    return (std::numeric_limits<T>::digits > std::numeric_limits<U>::digits) ? (t == static_cast<T>(u)) : (static_cast<U>(t) == u);
}

template <typename T, typename U>
constexpr bool cmp_equal_signed_unsigned(const T t, const U u) noexcept {
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(T);
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(U);
    return (t<T{ 0 }) ? false : (std::numeric_limits<T>::digits / 2 > std::numeric_limits<U>::digits) ? (t == static_cast<T>(u)) : (static_cast<U>(t) == u);
}
```

```

template <typename T, typename U>
constexpr bool cmp_less_same_sign(const T t, const U u) noexcept {
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(T);
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(U);
    return (std::numeric_limits<T>::digits > std::numeric_limits<U>::digits) ? (t < static_cast<T>(u)) : (static_cast<U>(t) < u);
}

template <typename T, typename U>
constexpr bool cmp_less_signed_unsigned(const T t, const U u) noexcept {
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(T);
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(U);
    return (t < T{ 0 }) ? true : (std::numeric_limits<T>::digits / 2 > std::numeric_limits<U>::digits) ? (t < static_cast<T>(u)) : (static_cast<U>(t) < u);
}

template <typename T, typename U>
constexpr bool cmp_less_unsigned_signed(const T t, const U u) noexcept {
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(T);
    ASSERT_INTEGRAL_NOT_BOOL_TYPE(U);
    return (u < U{ 0 }) ? false : (std::numeric_limits<U>::digits / 2 > std::numeric_limits<T>::digits) ? (static_cast<U>(t) < u) : (t < static_cast<T>(u));
}

#undef ERR_MSG_xxx_NEEDS_INTEGRAL_NOT_BOOL
#undef ASSERT_INTEGRAL_NOT_BOOL_TYPE
} // end details

/// Usage:
/// size_t i = ...
/// if(in_range<DWORD>(i)){
/// // safe to use i as a DWORD value, parameter...
/// } else {
/// // not possible to represent i as a DWORD
/// }
template <typename R, typename T>
constexpr bool in_range(const T t) noexcept {
    return std::is_unsigned<T>::value ? details::in_range_unsigned<R>(t) : details::in_range_signed<R>(t);
}

// equivalent of operator== for different types
/// Usage:
/// size_t i = ...
/// DWORD j = ...
/// if(cmp_equal(i,j)){
/// // i and j represent the same quantity
/// } else {
/// // i and j represents different quantities
/// }
template <typename T, typename U>
constexpr bool cmp_equal(const T t, const U u) noexcept {
    return
        (std::is_signed<T>::value == std::is_signed<U>::value) ? details::cmp_equal_same_sign(t, u) :
        (std::is_signed<T>::value) ? details::cmp_equal_signed_unsigned(t, u) : details::cmp_equal_signed_unsigned(u, t);
}

// equivalent of operator< for different integral types
/// Usage:
/// size_t i = ...
/// DWORD j = ...
/// if(cmp_less(i,j)){
/// // i < j
/// } else {
/// // i >= j
/// }
template <typename T, typename U>
constexpr bool cmp_less(const T t, const U u) noexcept {
    return
        (std::is_signed<T>::value == std::is_signed<U>::value) ? details::cmp_less_same_sign(t, u) :
        (std::is_signed<T>::value) ? details::cmp_less_signed_unsigned(t, u) : details::cmp_less_unsigned_signed(t, u);
}

```

VI. Effects on Existing Code

Since the proposed functions are not defined in any standard header, no currently existing code behavior will be changed.

VII. Design Decisions

Since there is no reason to compare `true` and `false` with other integral types, there isn't one to provide an overload for the `bool` integral type either.

The name of the functions (`cmp_equal`, `cmp_less` and others) are open to discussion, but the function names `std::less` and `std::greater` should not be used, since these do already exist, and have a different meaning.

VIII. Related Works

In 2016, Robert Ramey did a much bigger proposal (see [p0228r0](#)) regarding safe integer types. He also used similar functions proposed in this paper for implementing his classes and operators, therefore an alternative implementation can be found on his [github repository](#). This proposal addresses a smaller problem, namely comparing integral values, and is therefore much smaller.

The functions provided can be also used for creating safe integer types.

Another work, by Herb Sutter (see [p0515r0](#)), is about a new comparison operator (`<=>`). As far as I've understood the proposal the `operator<=>` should compare correctly different integral types, making part of this proposal obsolete if the operator is added to the language. While it would be a nice thing to have, having a new comparison operator that operates differently from the old operators may be counterintuitive and cause confusion, even if the new behaviour is more correct.